
Microsoft
STUDENT AMBASSADOR

Microsoft
Student Ambassadors

Orchestration using Airflow

Session 1

Our Aginda

1. History of sheduling
2. what’s airflow and it’s benifets
3. Core Concepts in airflow
4. Core components and Airflow arch
5. Set Up airflow
6. Our first dag

History of Scheduling

- Cron Origins
  - Born in Unix systems (1970s).
  - Named after Greek "chronos" (time).
  - Designed for periodic background jobs.

- Cron Limitations
  - No dependency management.
  - Limited visibility.
  - No automatic retries
  - Difficult to maintain at scale.

Let’s see how cron works with bash scripts

- Airflow
  - Developed at Airbnb (2014).
  - Python-based workflow platform.
  - Addresses all Cron limitations.

what’s airflow

- Airflow is an open-source platform to programmatically author,schedule and monitor workflows. (Smart to-do list for your data)

why airflow

- Organization: Sequence tasks, enforce dependencies, control timing
- Visibility: Track progress, detect issues, understand dependencies
- Flexibility & Scalability: Integrate tools, scale easily, customize workflows

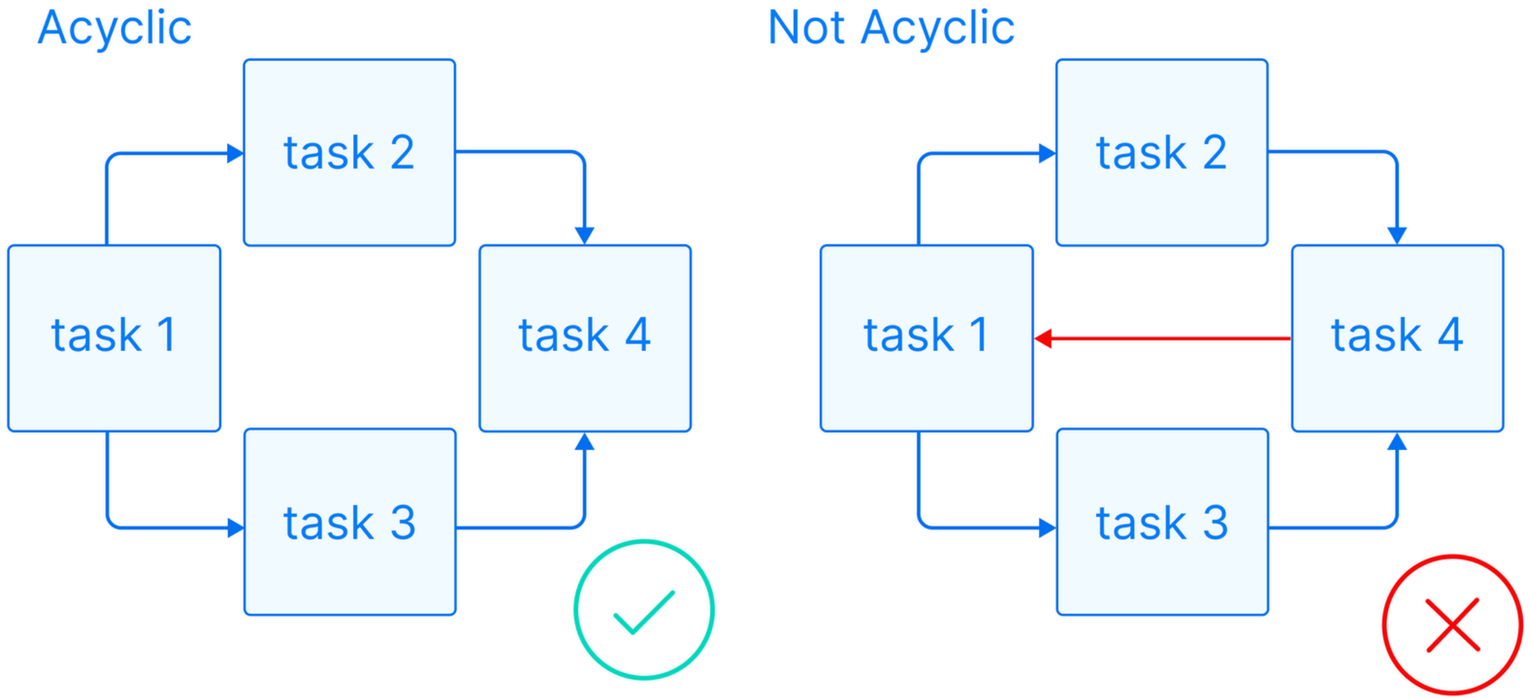
Benefits of Airflow :

- Dynamic & Python-Based: Flexible, easy to use, supports dynamic tasks & workflows, branching logic
- Scalable: Handles small to large workloads with multiple execution modes based on your setup
- User Interface: Visual dashboard to monitor, control, troubleshoot, and analyze workflows
- Extensible: Integrates with many tools (AWS, Snowflake, etc.) and supports custom features

Core Concepts of Apache Airflow

Concept 1: The DAG (Directed Acyclic Graph)

- A DAG (Directed Acyclic Graph) is a collection of all the tasks you want to run, organized in a way that clearly reflects their dependencies.
- It helps you define the complete structure of your workflow by showing exactly which tasks must be completed before others can start.



Core Concepts of Apache Airflow

Concept 2: Operator

- An Operator defines a single, ideally idempotent task in your DAG.Operators allow you to break down your workflow into discrete, manageable pieces of work.

Airflow comes with thousands of built-in Operators.
Here are some of the most commonly used ones:

- PythonOperator: Used to execute a Python function or script.
- BashOperator: Used to run Bash commands or scripts.
- SQLExecuteQueryOperator: Used to execute SQL queries against a database.
- FileSensor: Used to wait (poll) until a specific file appears.

Core Concepts of Apache Airflow

Types of Operators

1. Action Operators : Perform a specific action or run a particular type of job.

Examples:

- PythonOperator: Executes Python functions.
- BashOperator: Runs Bash commands.
- EmailOperator: Sends emails.

2. Transfer Operators : Facilitate data transfer between systems.

Examples:

- S3ToGCSOperator: Transfers files from Amazon S3 to Google Cloud Storage.
- MySqlToGoogleCloudStorageOperator: Moves data from a MySQL database to Google Cloud Storage.

Core Concepts of Apache Airflow

Types of Operators

3. Sensor Operators : Special operators that "wait" for a certain condition to be met before continuing execution.

Examples:

- FileSensor: Waits for a file to appear in a specified location.
- S3KeySensor: Waits for a specific object in an S3 bucket.

4. Trigger-Deferrable Operators (Airflow 2.x and Later):

Lightweight operators that yield control back to the scheduler until a specific condition is met.

Examples:

- TriggerDagRunOperator: Triggers another DAG to run.
- DeferrableFileSensor: A more efficient file sensor that does not block resources.

Also u can make ur custom operator for specialize need

Let’s see how to find the operators to use in my workflow

Core Concepts of Apache Airflow

Concept 3: Task / Task Instance

- A Task is a specific instance of an Operator. When you assign an Operator to a DAG and give it a unique task_id, it becomes a Task.
- Tasks are the actual executable units of work. When the DAG runs, these tasks are what get executed (either sequentially or in parallel).

Concept 4: Workflow

- A Workflow is the entire process defined by your DAG. It includes all the tasks, their dependencies, and the overall logic of the data pipeline.
- It represents your complete data pipeline — showing how all the individual pieces work together to achieve your end goal.

```
DAG describes the workflow, while Workflow is the actual running process of that DAG.
```

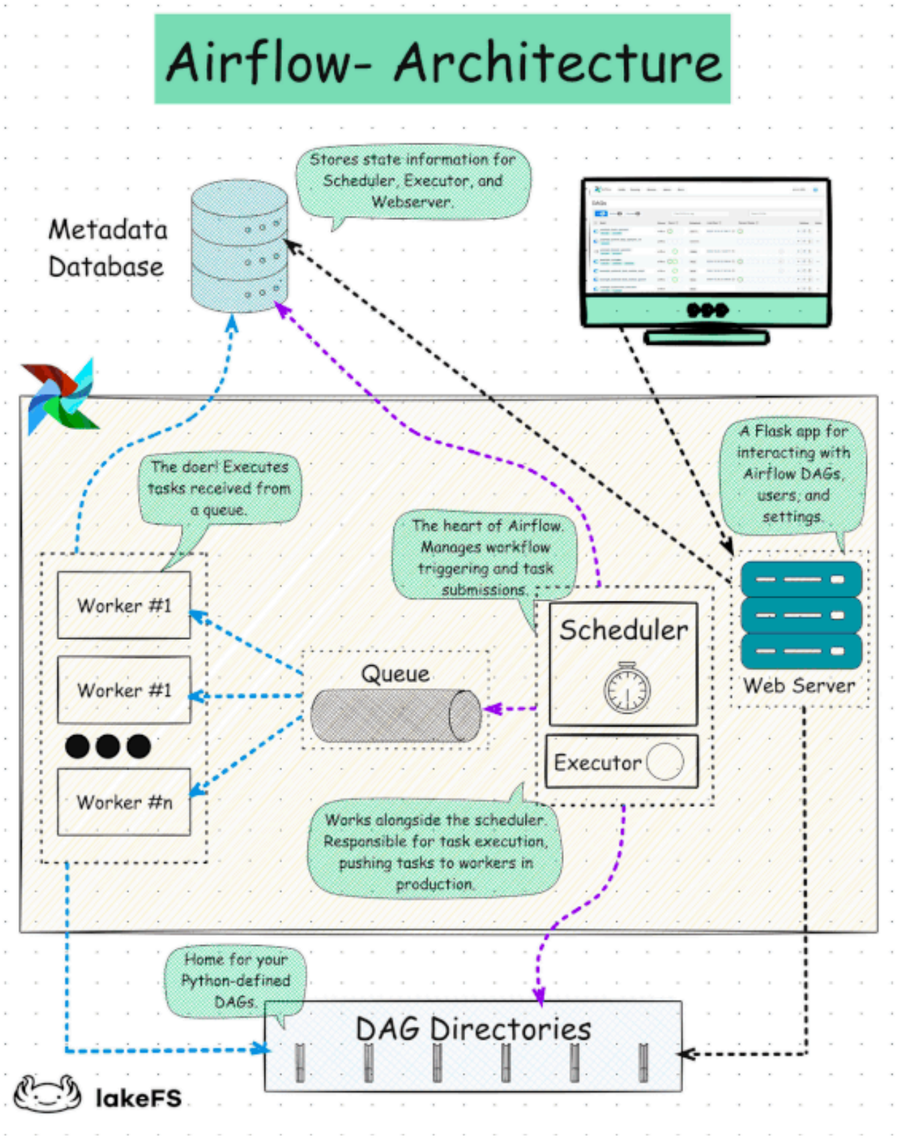
Core Components of Apache Airflow

Component 1: The Metadata Database

- This is a database that stores information about your tasks, their status, execution history, and all related metadata. It keeps track of everything important about your workflows, including DAG runs, task instances, logs, and scheduling details.

Component 2: The Scheduler

- The Scheduler is responsible for determining when tasks should run and in what order. It continuously checks the metadata database, respects task dependencies, and triggers task execution according to the DAG schedule.



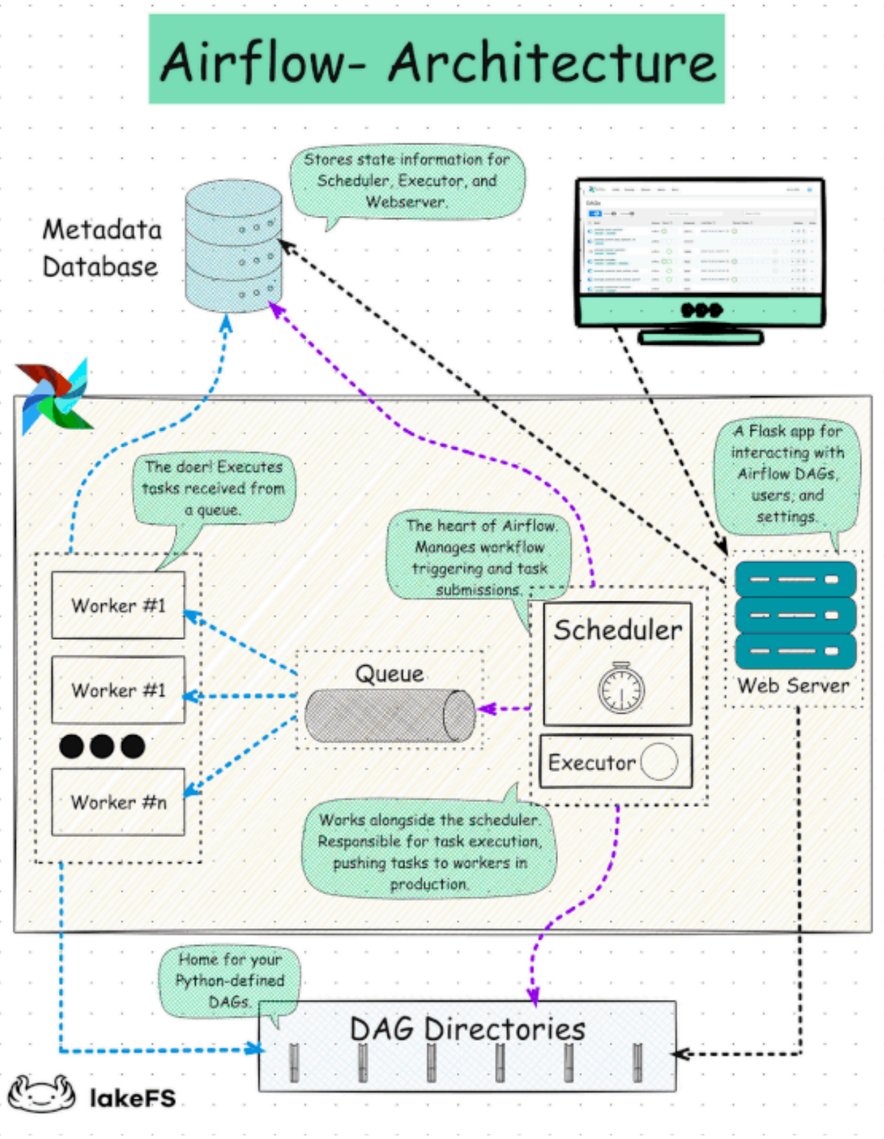
Core Components of Apache Airflow

Component 2+: The DAG File Processor

- The DAG File Processor parses DAG files and serializes them into the metadata database. It used to be part of the Scheduler, but starting from Airflow 3.0, it became a separate component for better scalability and security

Component 3: The Executor

- The Executor determines how your tasks will be run. It manages the execution strategy — deciding whether to run tasks sequentially or in parallel, and on which system (Local, Celery, Kubernetes, etc.).



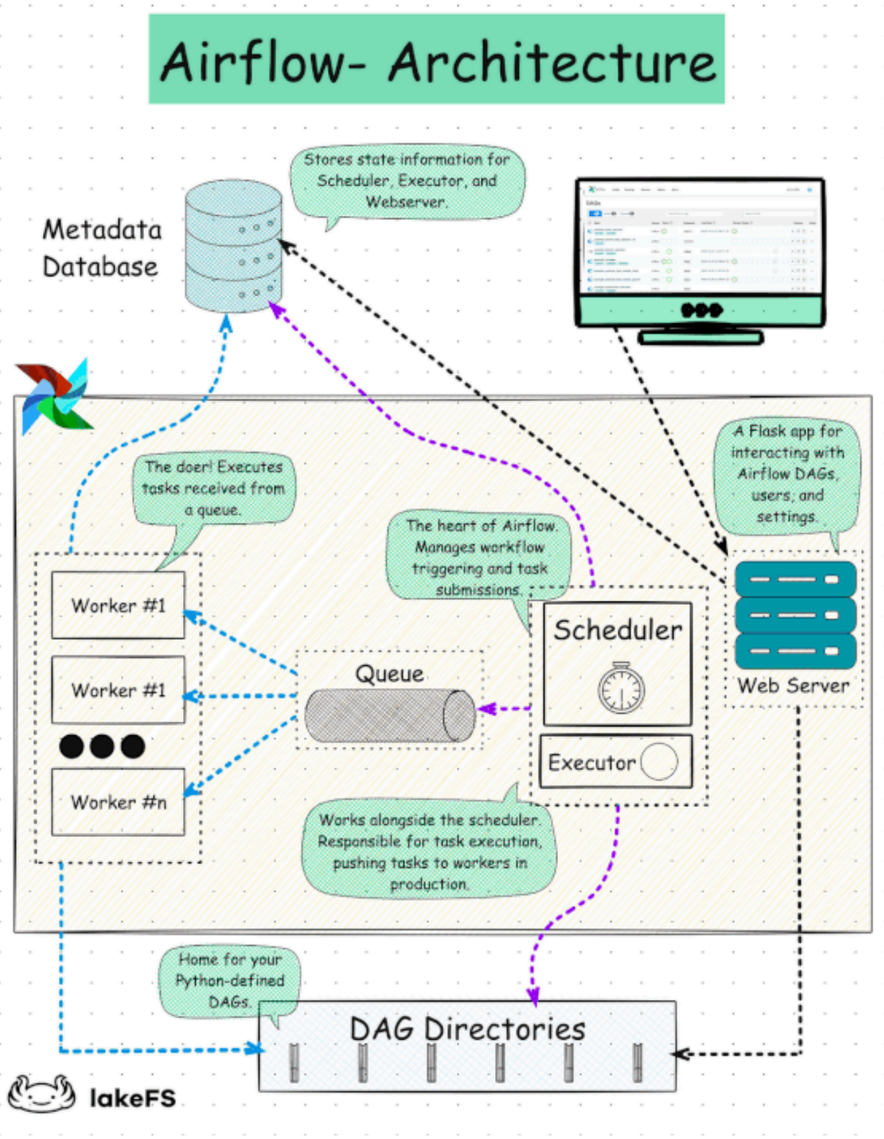
Core Components of Apache Airflow

Component 4: The API Server

- The API Server provides endpoints for task operations and serves the Airflow Web UI. It handles requests from the Executor and allows users to view, manage, and monitor workflows through a web browser.

Component 5: The Worker

- Workers are the processes that actually execute the tasks. They perform the real work defined in your tasks, such as running Python code, executing SQL queries, calling APIs, or transferring files.



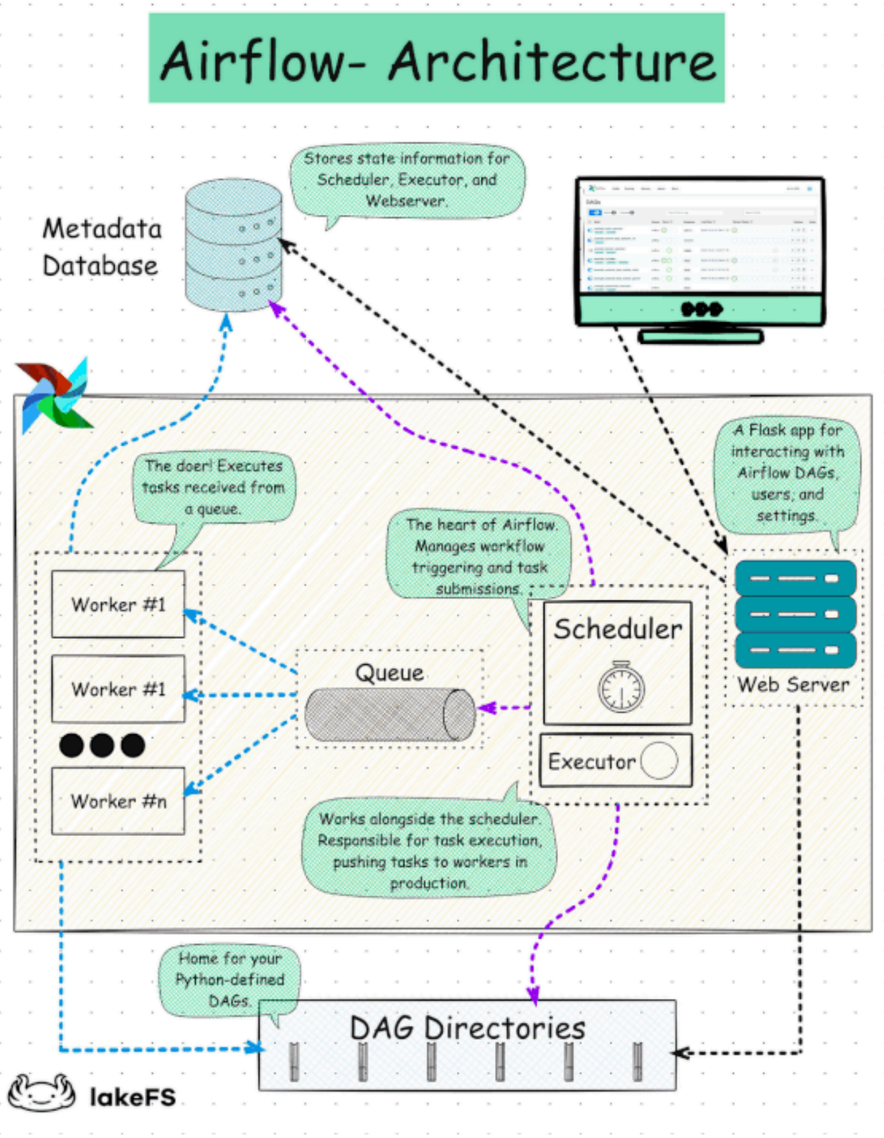
Core Components of Apache Airflow

Component 6: The Queue

- The Queue is a list of tasks waiting to be executed. It helps manage the order of task execution, especially when there are many tasks running concurrently (commonly used with Celery Executor).

Component 7: The Triggerer

- The Triggerer is responsible for managing deferrable tasks — tasks that wait for external events (such as sensors, file arrival, or API responses). It allows Airflow to efficiently handle long-waiting tasks without blocking the Scheduler or wasting resources.



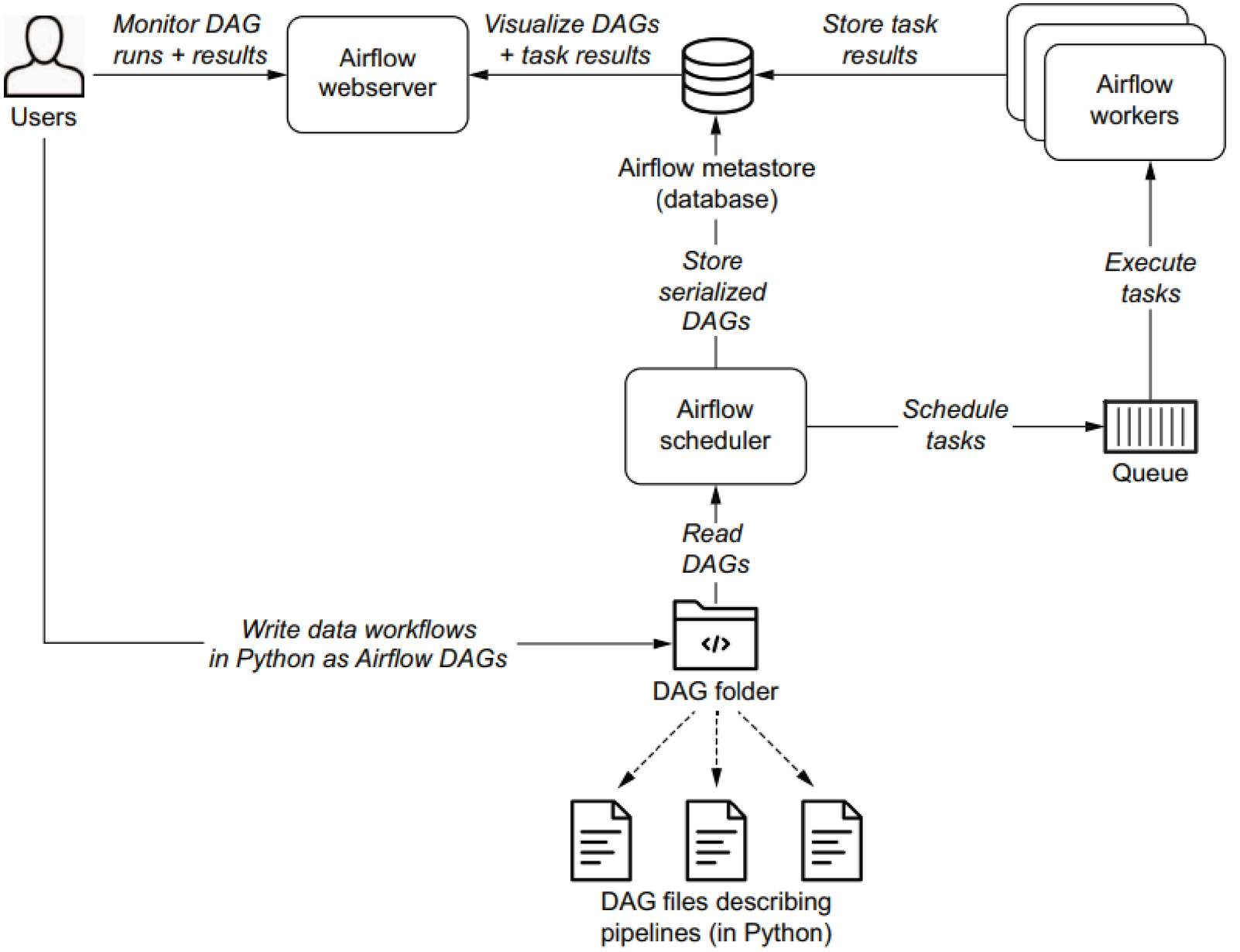


Figure 1.8 Overview of the main components involved in Airflow (e.g., the Airflow webserver, scheduler, and workers)

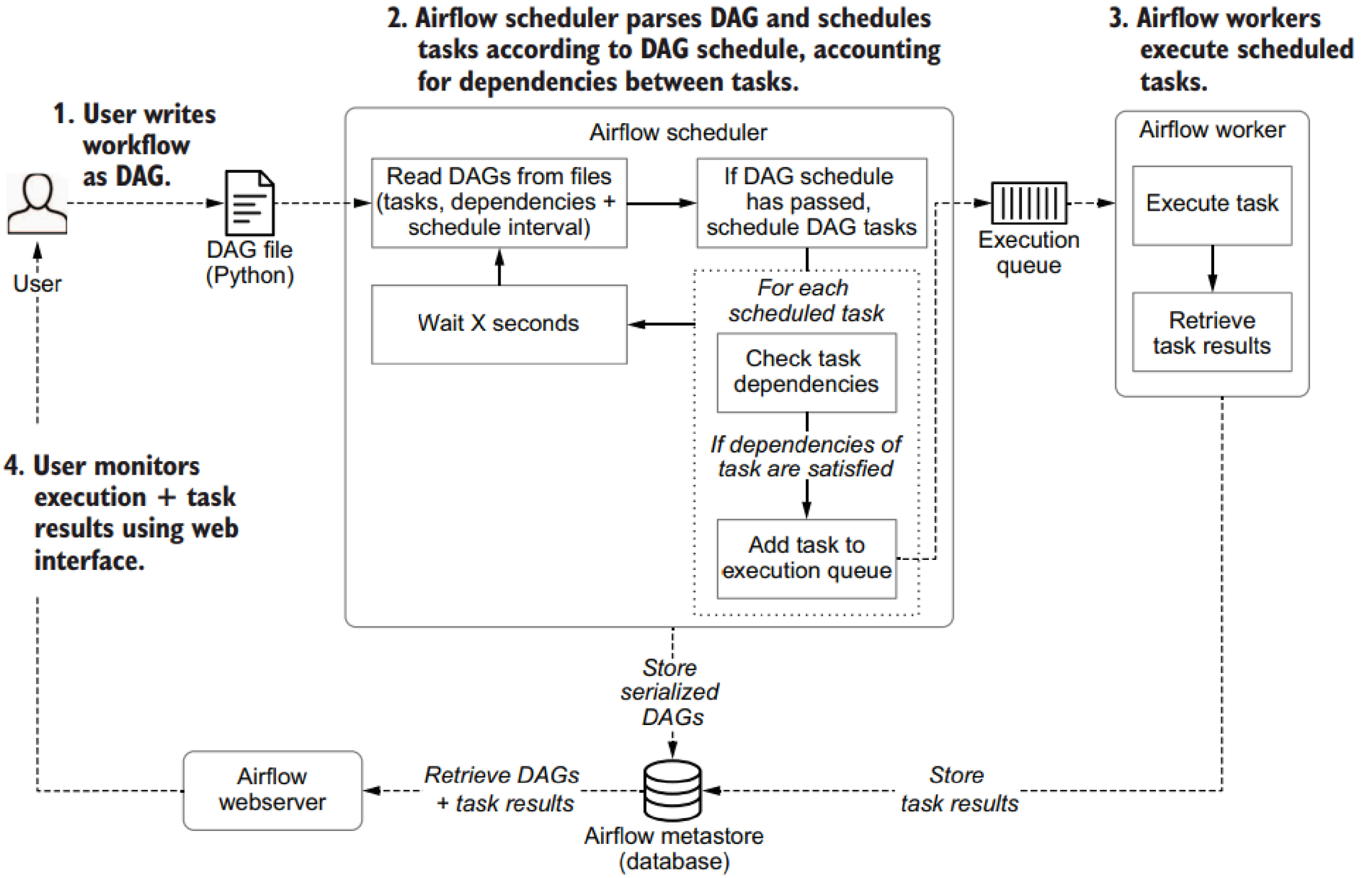
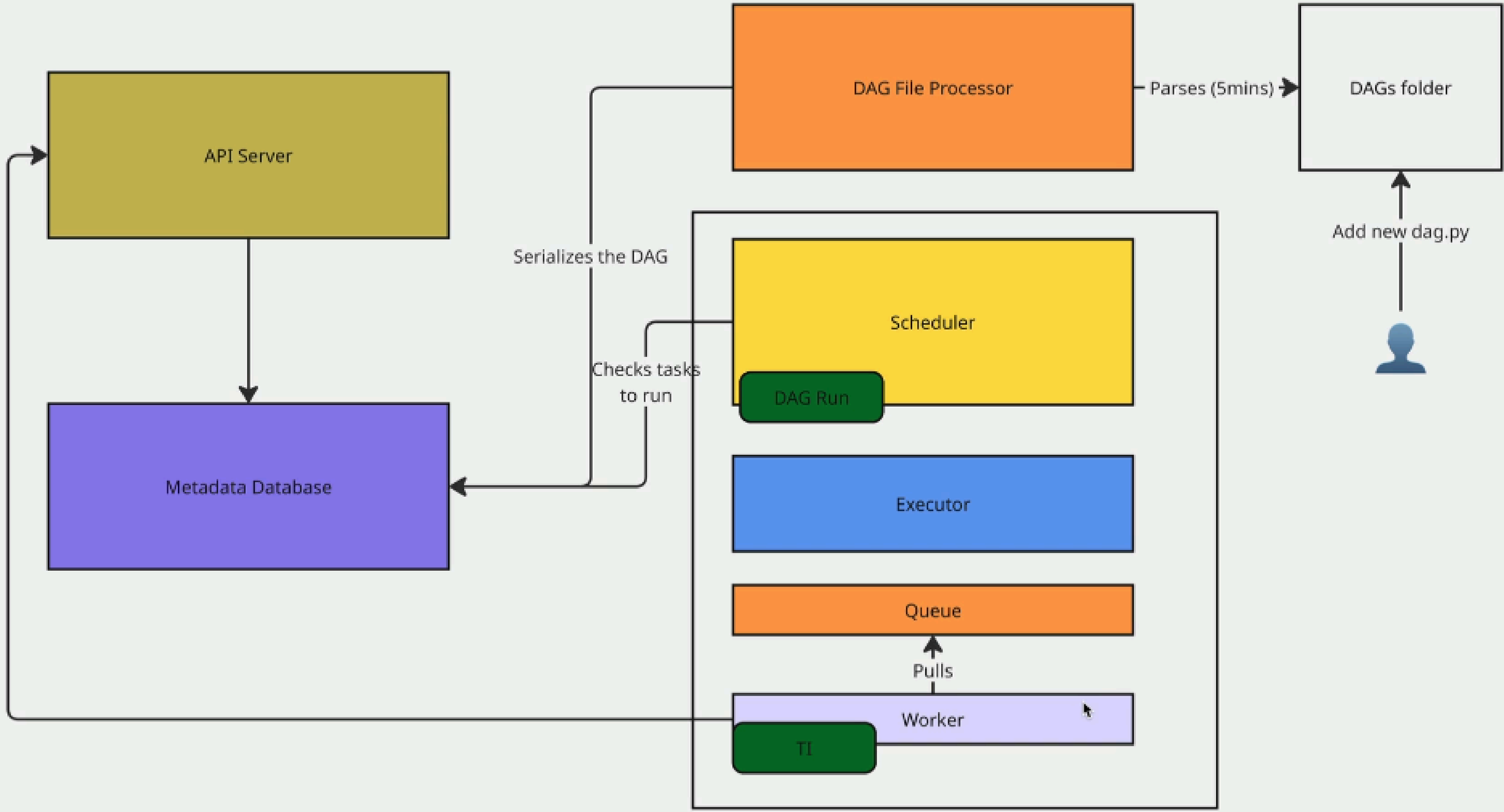
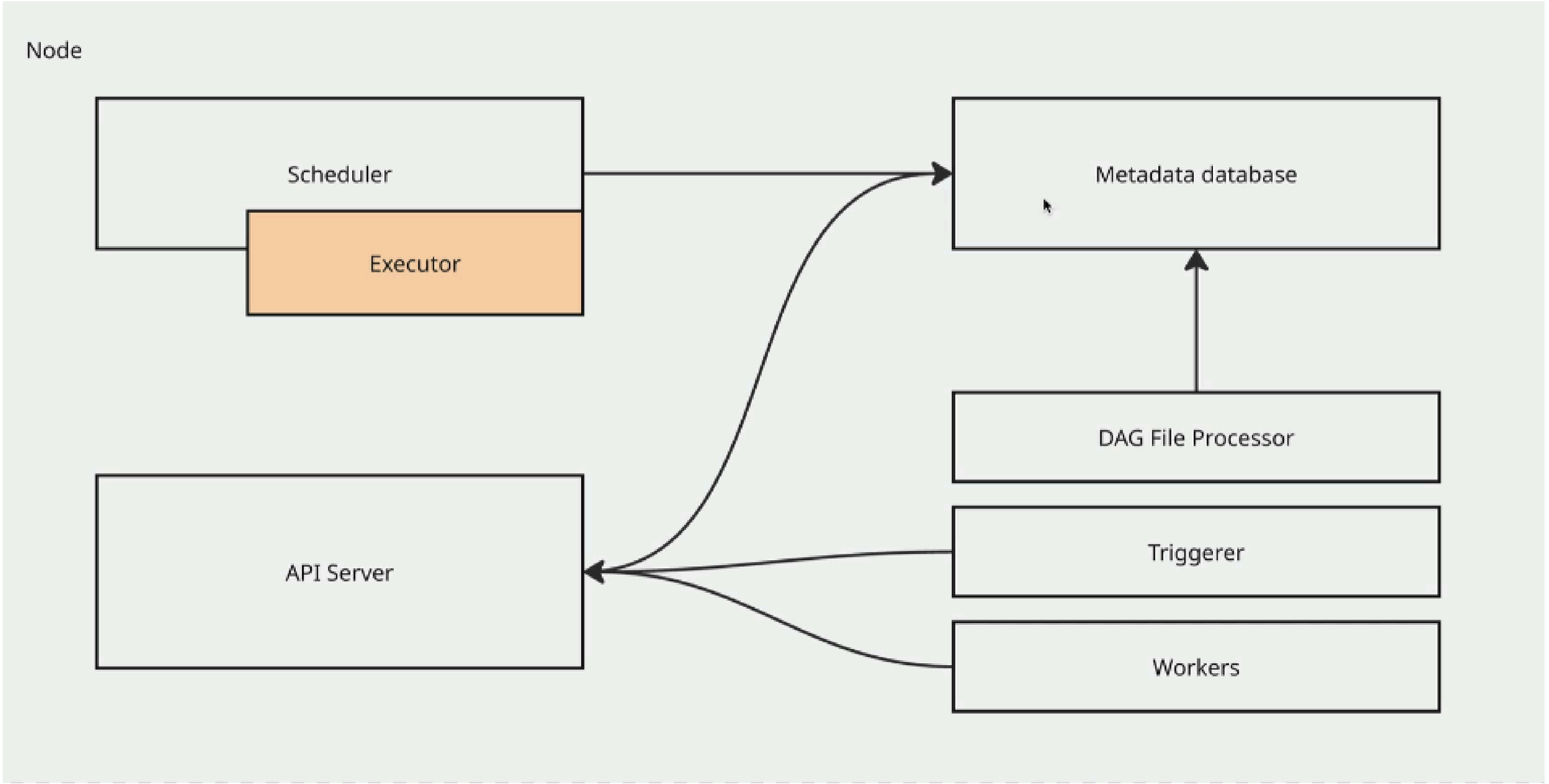


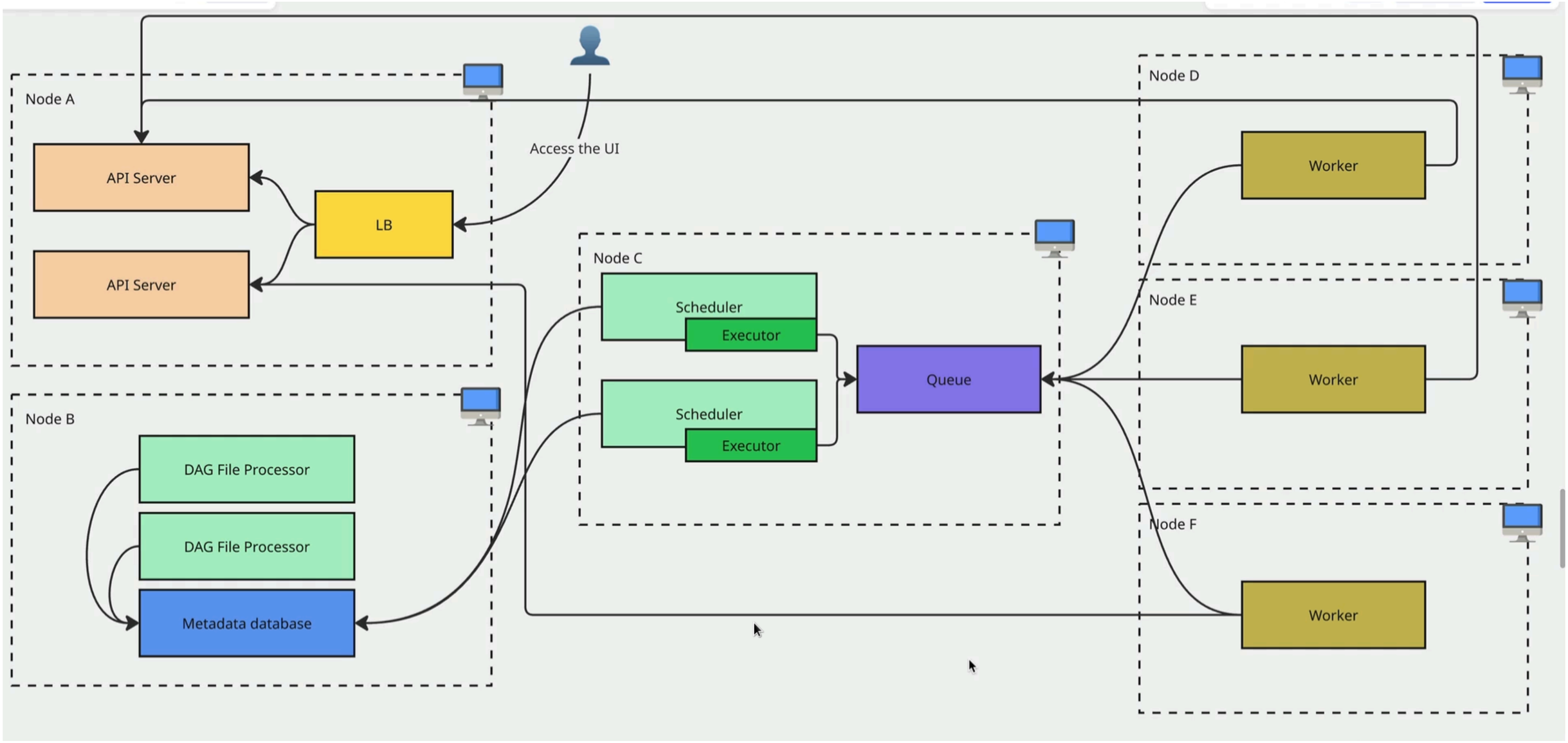
Figure 1.9 Schematic overview of the process involved in developing and executing pipelines as DAGs using Airflow


DAG File Processor
Parses (5mins)
DAGs folder
API Server
Serializes the DAG
Add new dag.py
Scheduler
Checks tasks
to run
DAG Run
Metadata Database
Executor
Queue
Pulls
Worker
TI

Single node arch



Multi nodes arch



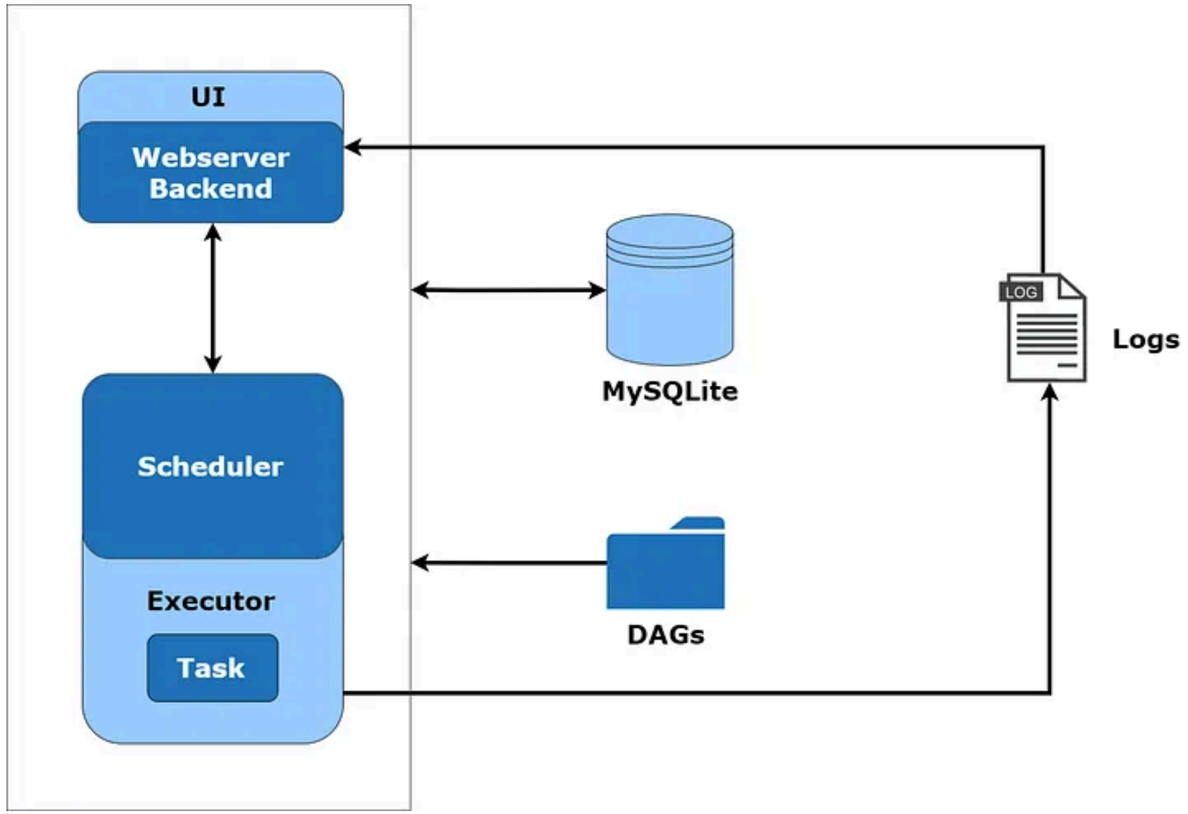
Let’s see how to SetUp Apache Airflow

Demo

Types of Executors in Apache Airflow

Sequential Executor

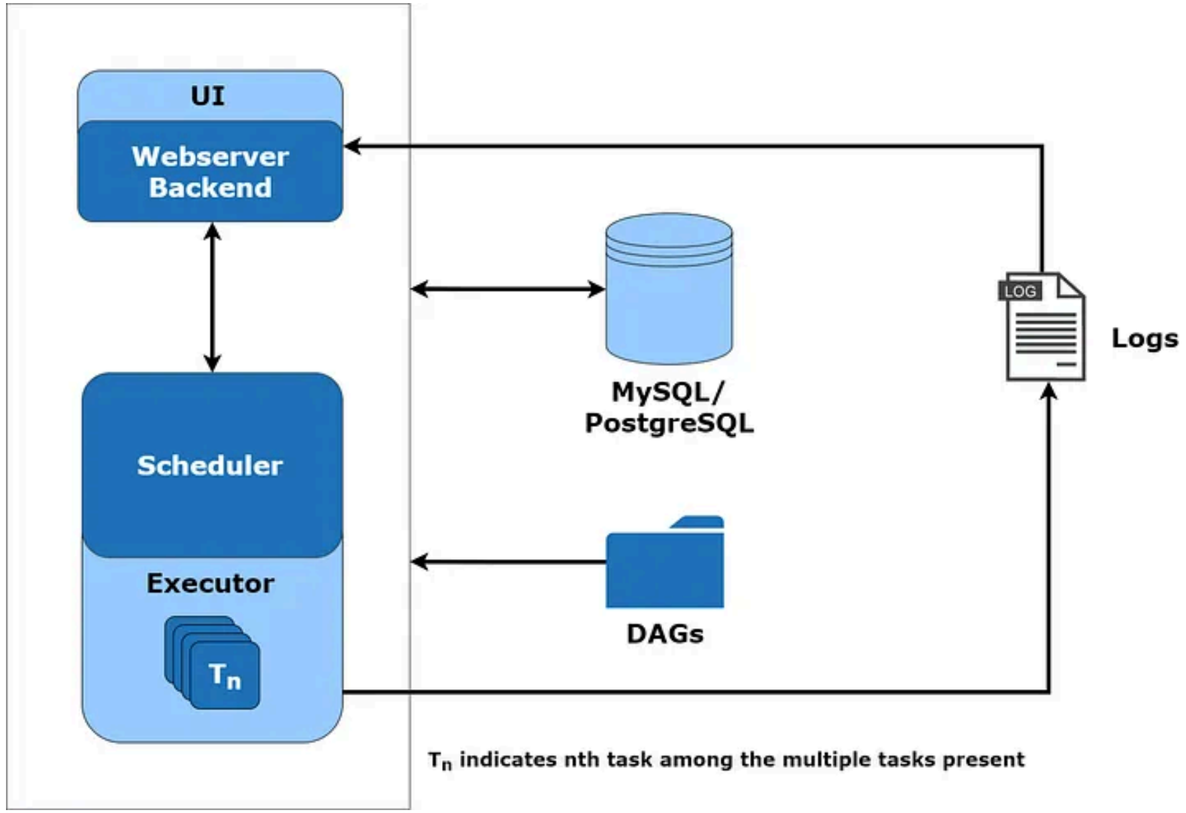
- It is a lightweight local executor, which is available by default in airflow. It runs only one task instance at a time and is not production-ready.
- It can run with SQLite since SQLite does not support multiple connections.
- It is prone to single-point failure, and we can utilize it for debugging purposes.



Types of Executors in Apache Airflow

Local Executor

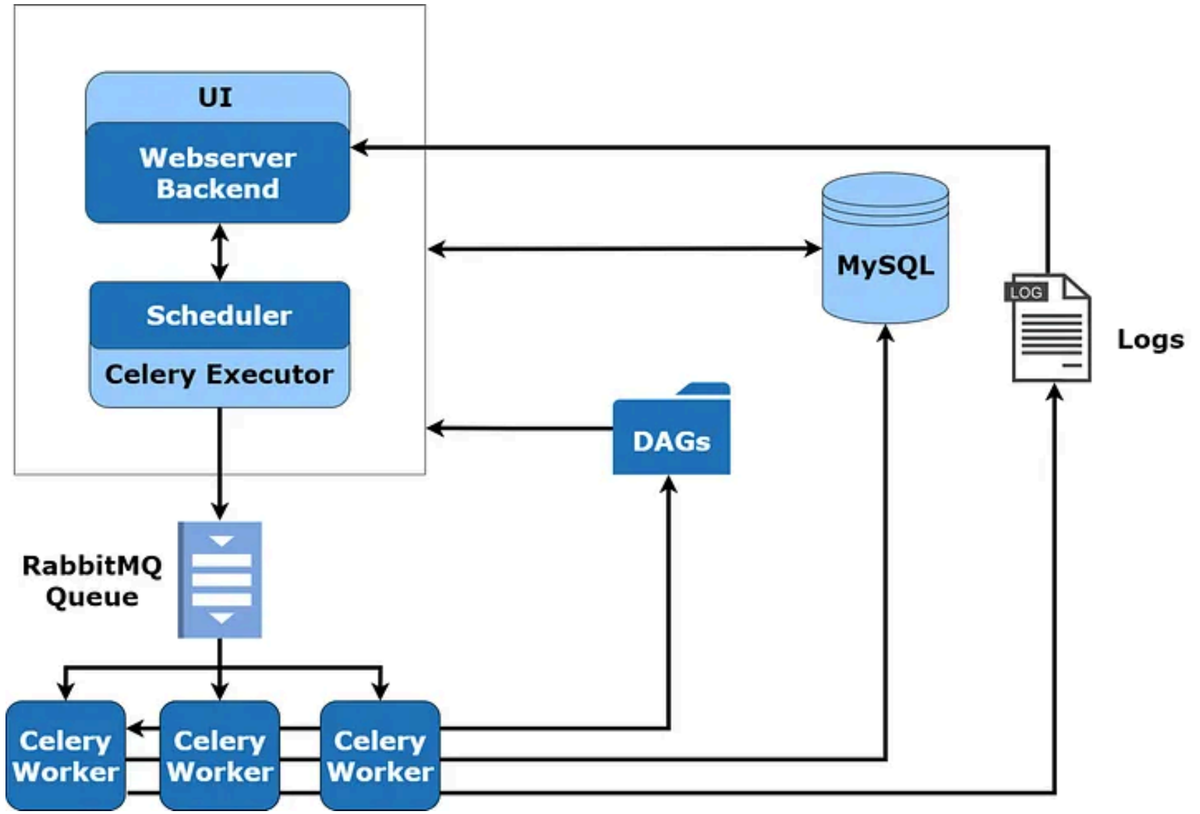
- Unlike the sequential executor, the local executor can run multiple task instances.
- Generally, we use MySQL or PostgreSQL databases with the local executor since they allow multiple connections, which helps us achieve parallelism.



Types of Executors in Apache Airflow

Celery Executor

- Celery is a task queue, which helps in distributing tasks across multiple celery workers. The Celery Executor distributes the workload from the main application onto multiple celery workers with the help of a message broker such as RabbitMQ or Redis. The executor publishes a request to execute a task in the queue, and one of several worker nodes picks up the request and runs as per the directions provided.



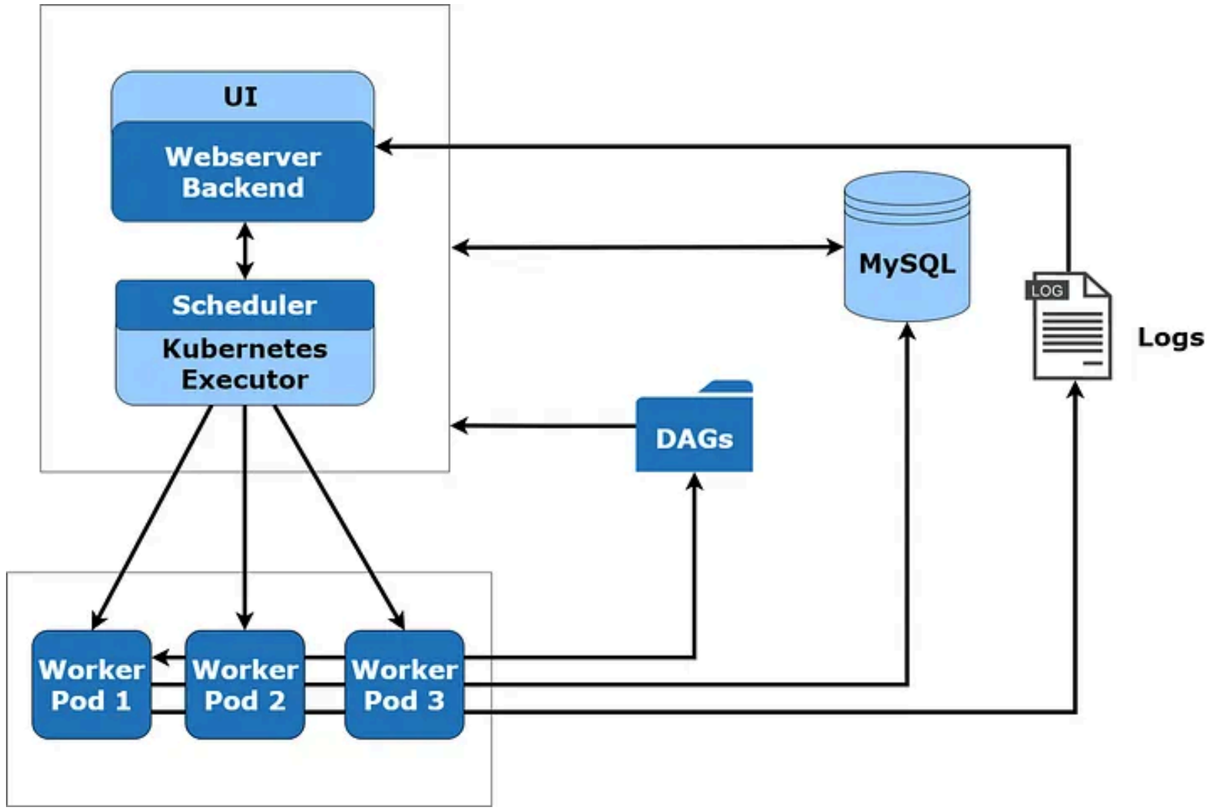
- RabbitMQ is a message broker and provides communication between multiple services by operating message queues. It also offers an API for other services to publish and subscribe to the queues.

Types of Executors in Apache Airflow

Kubernetes Executor

- The Kubernetes Executor uses the Kubernetes API for resource optimization. It runs as a fixed-single Pod in the scheduler that only requires access to the Kubernetes API.

- A Pod is the smallest deployable object in Kubernetes.



- When a DAG submits a task, the Kubernetes Executor requests a worker pod. The worker pod is called from the Kubernetes API, which runs the job, reports the result, and terminates once the job gets finished

References

- https://www.udemy.com/course/the-complete-hands-on-course-to-master-apache-airflow/
- https://airflow.apache.org/docs/
- https://www.astronomer.io/ebooks/data-pipelines-with-apache-airflow/

Thanks